

Routing Solve: Unassigned-Rider Analysis

2026-09-20 → 2026-09-26 · 7 service dates · source: run_ml_solver_db.py (live DB, engine=osrm, use_ml=True)

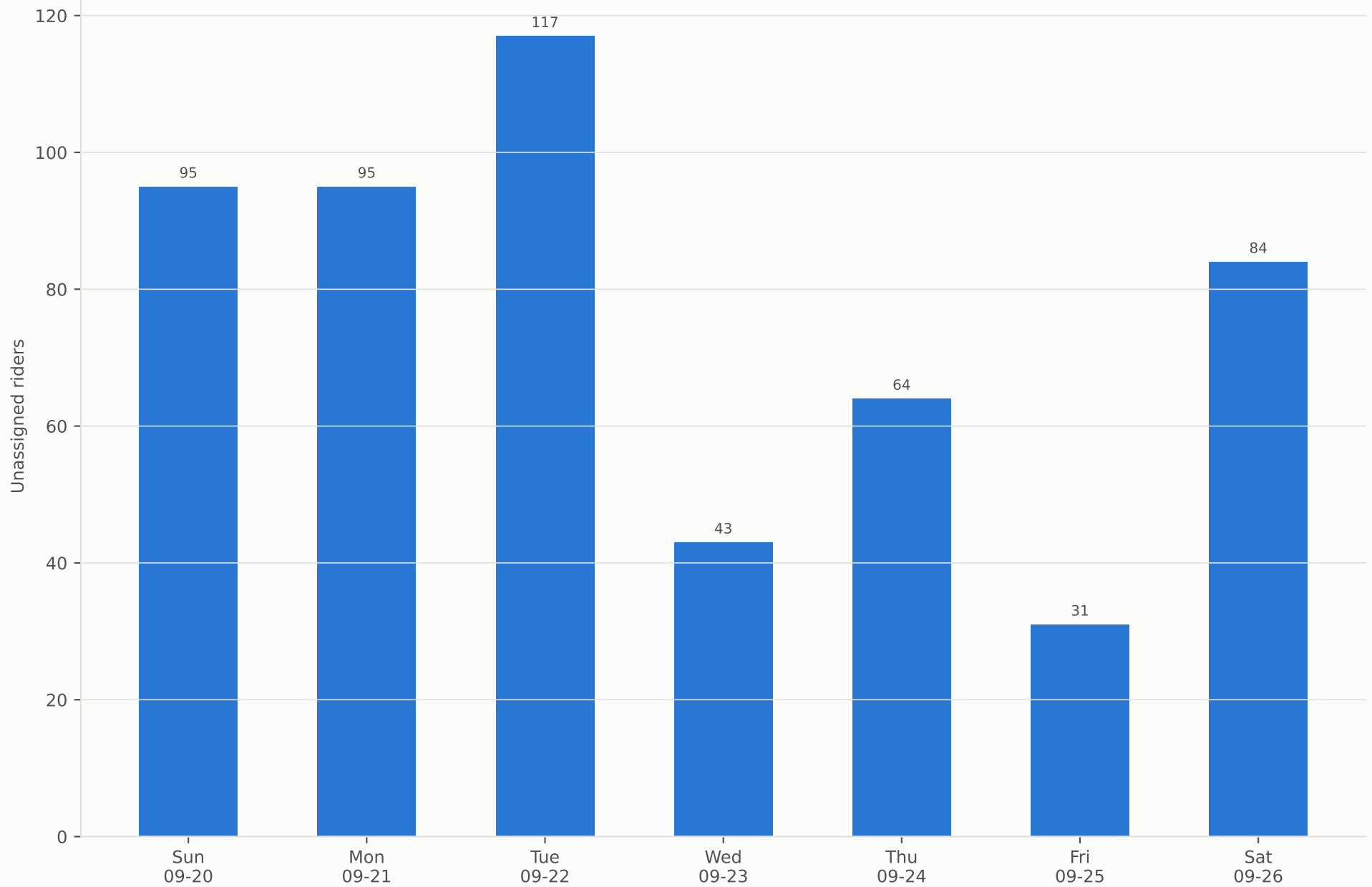
Date	Day	Pickup req	Dropoff req	Total req	Routed	Unassigned	Unassigned %
2026-09-20	Sun	302	668	970	875	95	9.8%
2026-09-21	Mon	290	668	958	863	95	9.9%
2026-09-22	Tue	301	683	984	867	117	11.9%
2026-09-23	Wed	288	653	941	898	43	4.6%
2026-09-24	Thu	297	663	960	896	64	6.7%
2026-09-25	Fri	295	693	988	957	31	3.1%
2026-09-26	Sat	282	652	934	850	84	9.0%

Week totals: 6735 requests → 6206 routed, 529 unassigned (7.9%)

Notes on this data:

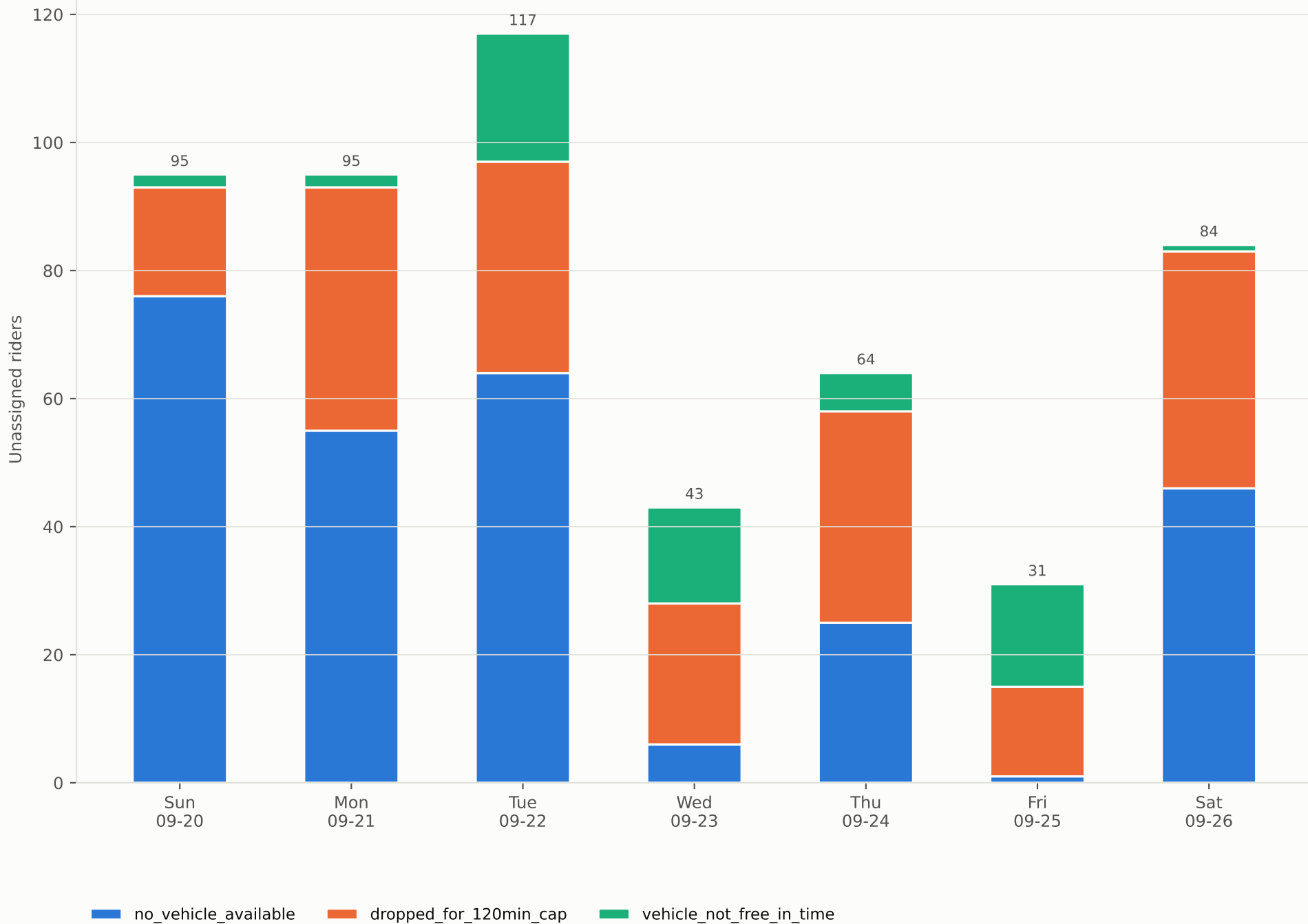
- Every request across all 7 dates has valid coordinates (no 'no_coordinates' unassigned) -- unlike the offline test fixture, this live data has no missing-location rows.
- 2026-09-22 was solved twice back-to-back (identical inputs, live DB + real OSRM). unassigned matched exactly both times (117), but routes differed (189 vs 190, pickup_routes 67 vs 68) -- evidence the solve is not perfectly reproducible run-to-run (see Statistical Analysis).

Total unassigned riders per service date

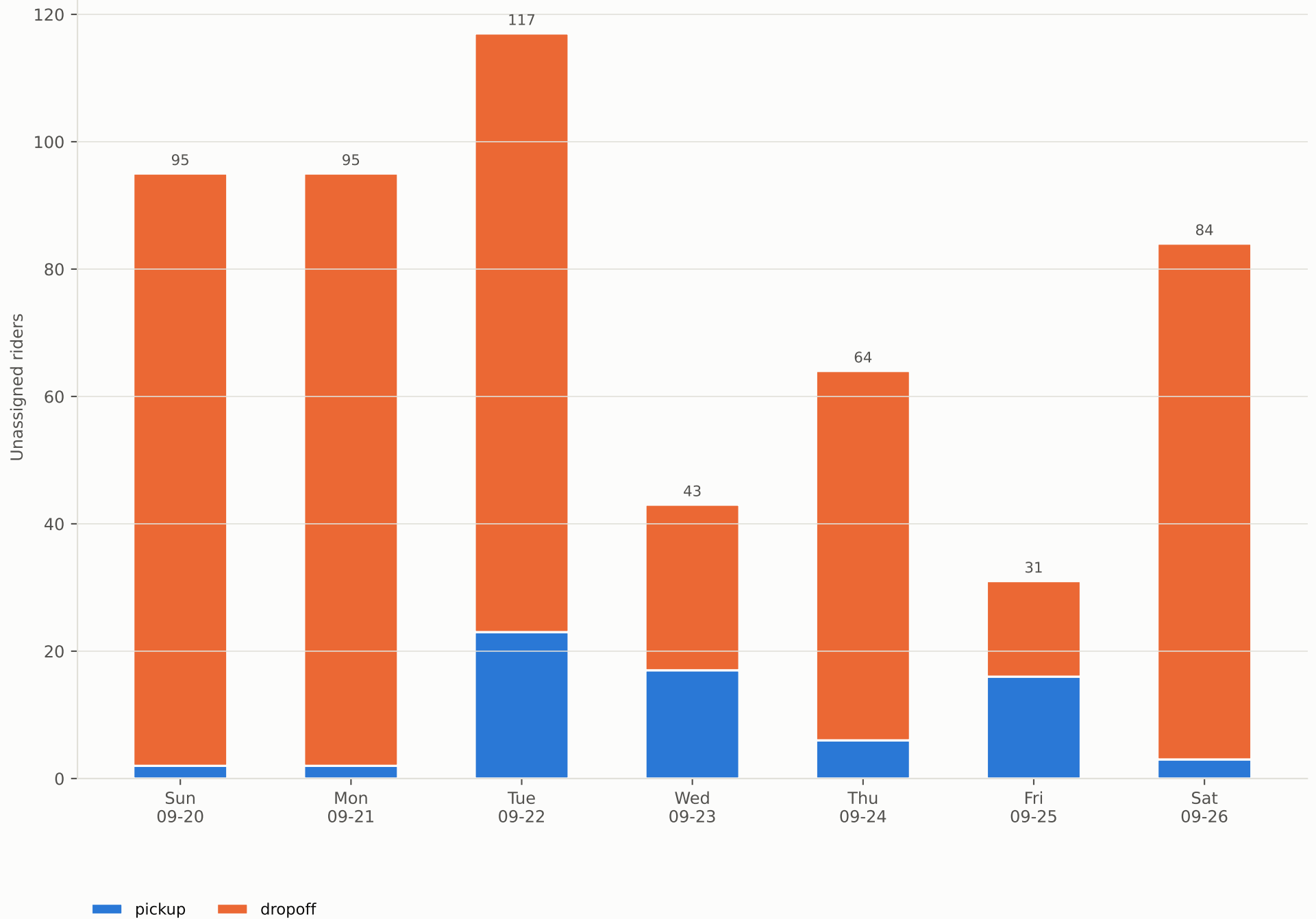


Range: 31-117 · mean 75.6 · stdev 28.6

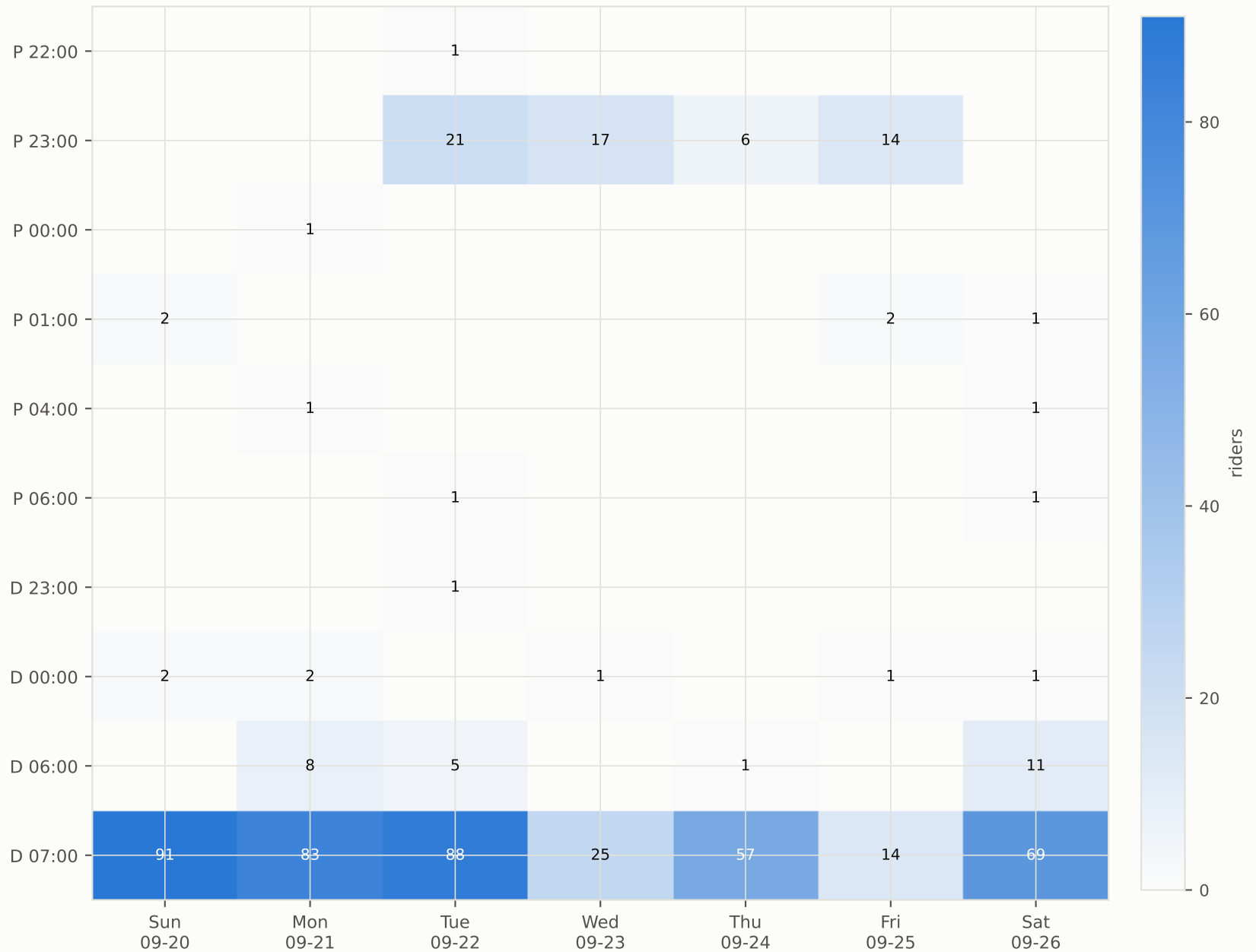
Unassigned riders by reason, per service date



Unassigned riders by request type, per service date



Unassigned riders by (type, shift) across the week

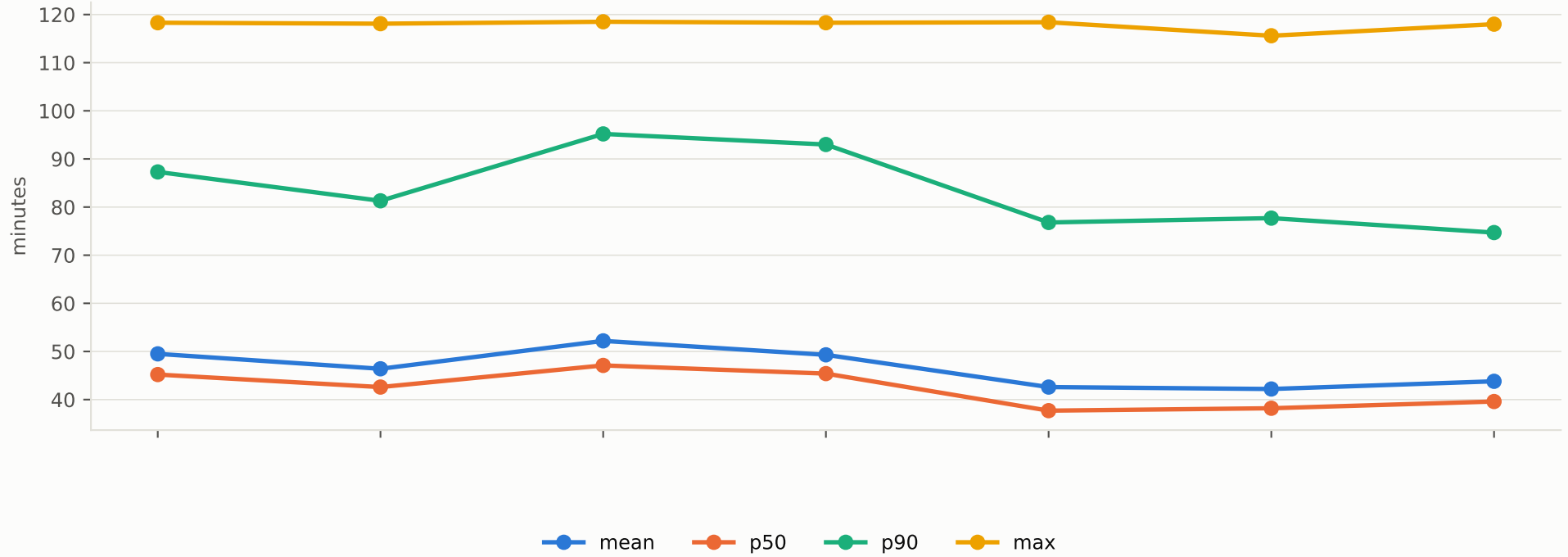


Unassigned rate over the week

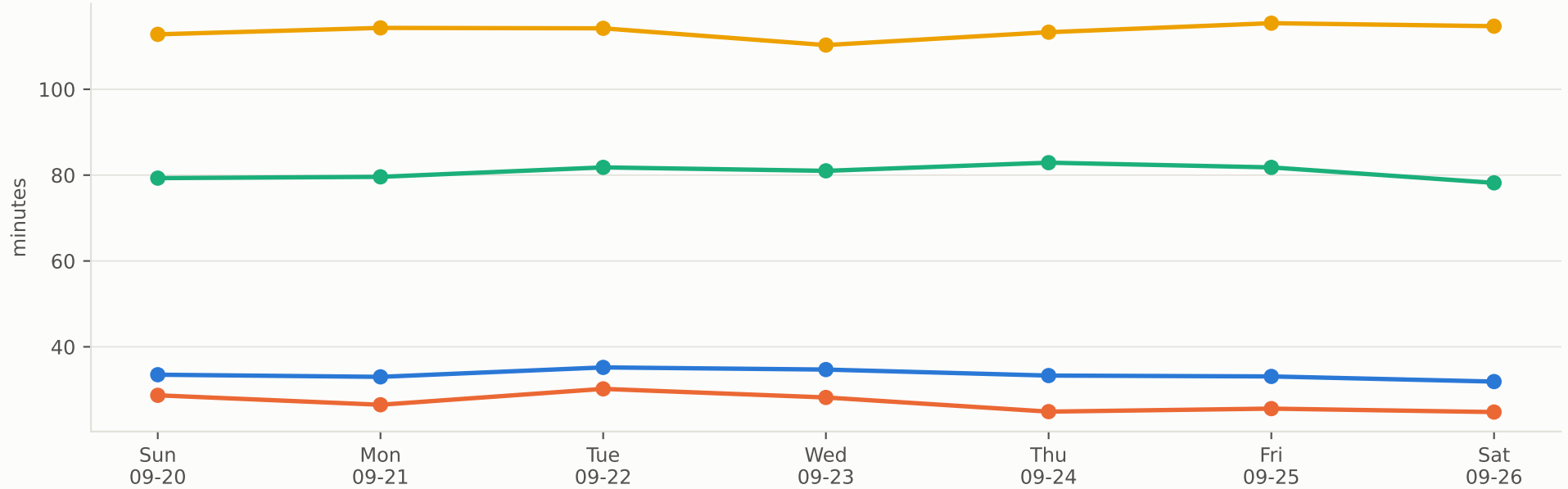


Passenger ride-time fairness across the week

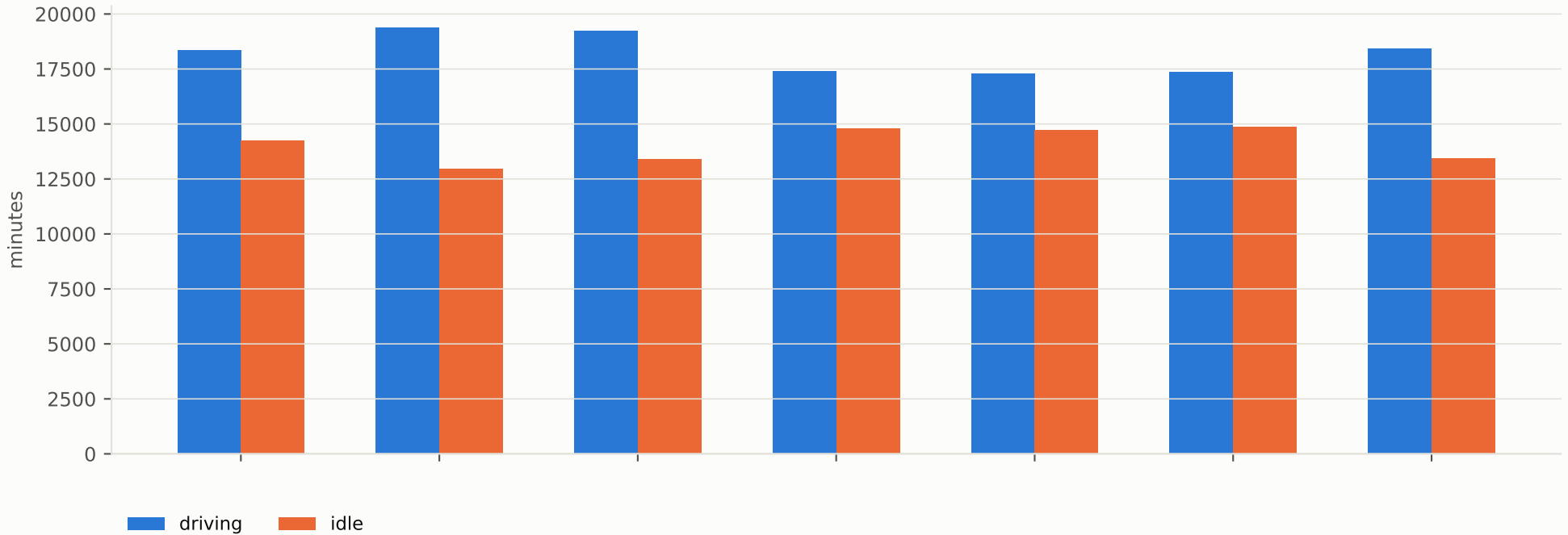
Pickup ride time in the car



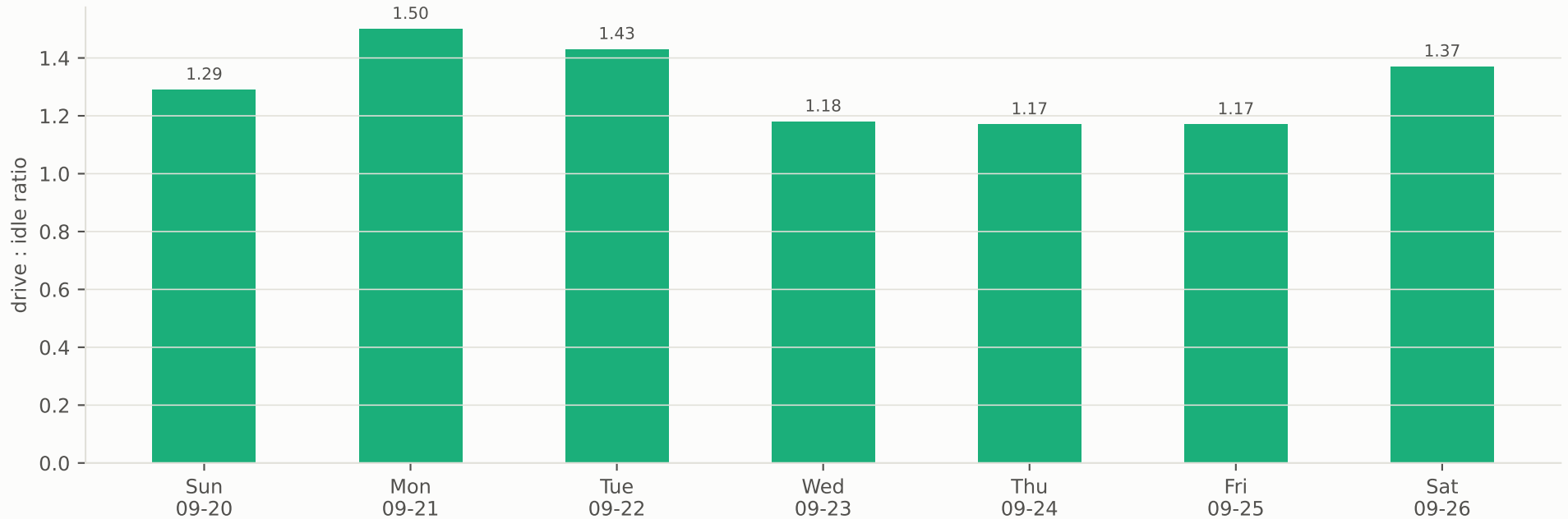
Dropoff ride time in the car



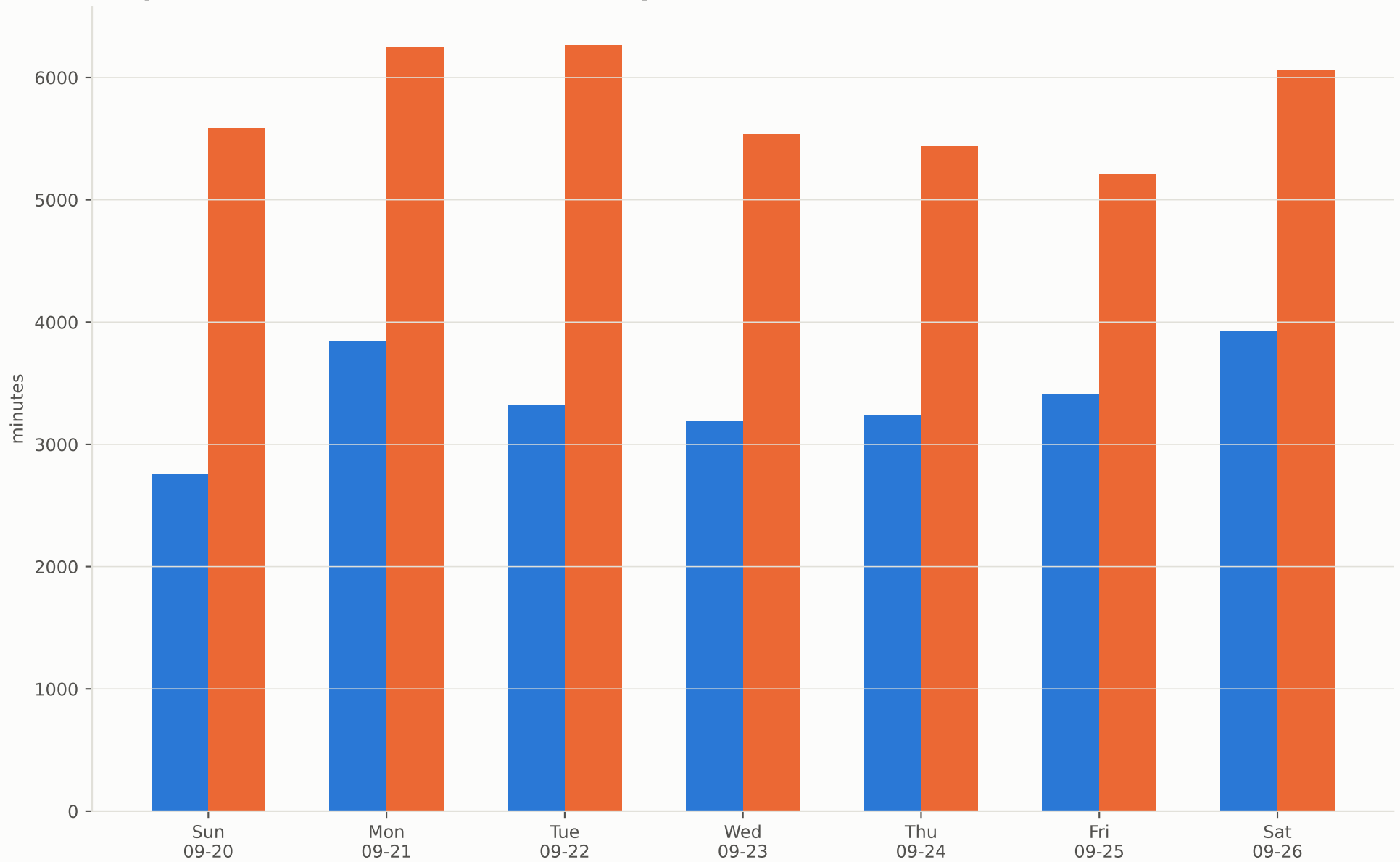
Fleet driving vs idle minutes, per night



Drive:idle ratio, per night (higher = fleet works harder per idle minute)



Drop-off deadhead minutes: driven vs priced-but-not-driven



■ deadhead IN (driven)
■ return legs (not driven unless reused)

Statistical Analysis & Interpretation

Descriptive statistics (n = 7 service dates)

- Unassigned count: mean 75.6, median 84.0, stdev 28.6, range 31-117.
- Unassigned rate: mean 7.9%, range 3.1% (2026-09-25) to 11.9% (2026-09-22).
- Total requests/night ranged 934-988 (mean 962); dropoff requests consistently outnumber pickups 669 vs 294 on average (~2.3x).

Correlation (n=7 -- descriptive only, not a hypothesis test)

- dropoff_requests vs unassigned count: $r = -0.03$
- total_requests vs unassigned count: $r = 0.08$
- With only 7 dates this is suggestive, not conclusive -- but the direction is consistent with the reason breakdown below: nights with more dropoff demand packed into 07:00 see more 'no_vehicle_available' at that shift specifically.

Dominant failure mode

- no_vehicle_available: 273 riders across the week (52% of all unassigned)
- dropped_for_120min_cap: 194 riders across the week (37% of all unassigned)
- vehicle_not_free_in_time: 62 riders across the week (12% of all unassigned)
- Overall dominant reason: 'no_vehicle_available' -- but its lead over 'dropped_for_120min_cap' is data-dependent (see per-day breakdown, page 3): it wins 5 of 7 days but 'dropped_for_120min_cap' actually leads on 2026-09-22 and 2026-09-24.

Where unassigned riders concentrate

- The 07:00 drop-off shift alone accounts for 273 'no_vehicle_available' + 154 'dropped_for_120min_cap' riders across the week -- by far the single biggest concentration, and consistent every single day (dominant on all 7 dates, see heatmap page 5).
- The 23:00 pickup shift's 'vehicle_not_free_in_time' totals 55 riders, but is not universal -- it appears on 4 of 7 dates and is exactly 0 on 3, so it is a scheduling-tightness effect that depends on how the night's earlier events line up, not a fixed structural gap like the 07:00 drop-off.

Reproducibility

- 2026-09-22 was solved twice, back-to-back, against the same live DB rows. 'unassigned' matched exactly (117 both times, identical reason/shift breakdown), but 'routes' (189 vs 190) and 'pickup_routes' (67 vs 68) did not.

- Interpretation: which riders get left behind is stable, but exactly how the solver merges/splits routes among the surviving riders is not -- consistent with the DB adapter issuing no explicit ORDER BY (see run_ml_solver_db.py's RoutingAdapter queries), so tie-breaks that depend on row order can flip between otherwise-identical runs.

What this suggests for tuning (see prior hyperparameter discussion)

- The 07:00 drop-off bottleneck is dominated by 'no_vehicle_available', which SolverConfig knobs (near_tie_slack, cluster_*, walk_limit_min) do not touch at all -- it is gated by _can_serve_dropoff's free-time/deadhead check, so the real levers are fleet size and how long EARLIER trips tie up each car (which loops back to the ML duration model's accuracy, since it sets every _free_at timestamp for the night).
- 'dropped_for_120min_cap' at 07:00 is the one bucket directly gated by max_route_minutes/boarding_buffer_min -- the two knobs with a mechanical, guaranteed effect on this specific number.
- 23:00 pickup's 'vehicle_not_free_in_time' is gated by office_buffer_min and by how tightly earlier shifts packed the fleet -- worth checking on the specific dates where it spikes (09-22, 09-23, 09-25) rather than assuming a single fix applies every night.